Skaters on a River: Calibrated Forecast Features for Streaming Regression, with a Pathwise Regret-Transfer Theorem

Peter Cotton
Micropediction
peter.cotton@microprediction.com
https://ice-skaters.microprediction.org

July 8, 2026

Abstract. The Python package **ice-skaters** replaces each numeric stream entering a streaming machine-learning pipeline with two scalars produced by the stream's own online distributional forecaster: the one-step predictive mean, which carries the level, and the standardized surprise, the probability integral transform of the arriving value pushed through the standard normal quantile, which carries the news and is bounded near $|z| = 7$ by construction. The package ships two estimators compatible with the **river** streaming-learning protocol: a transformer for input streams and a regressor wrapper that adds the target's own pair while leaving the target raw. On **river**'s own regression datasets under **river**'s progressive-validation protocol, the wrapper alone beats the recommended `StandardScaler` pipeline on three of four datasets untouched and ten seeds out of ten under simulated feature contamination, while tying the fourth; in controlled simulation the full substitution beats raw features, a running z-score, a median/MAD winsorizer, and a Huberised loss thirty seeds out of thirty under every contamination type tested, at a small clean-data toll. We report the losing rows with the winning ones: distance-based learners do not benefit, entity-interleaved streams handicap a single forecaster per key, informative heavy tails should not be tamed, and the front-end costs roughly 900× a running scaler per tick. The theoretical contribution is a pathwise regret-transfer theorem with measured constants and no calibration assumption: conjugated through the forecaster's predictive CDF, an online linear refinement of the surprises achieves log-loss within an explicitly computable $O(d \log T)$ of every refinement in its class, on any data sequence whatsoever, and in particular can never lose more than $O(d \log T)$ nats to the forecaster alone.

Keywords: online learning, distributional forecasting, probability integral transform, robust regression, streaming machine learning, regret bounds, Python, **ice-skaters**.

1 Introduction

A streaming regression consumes raw numbers it has no right to trust. The standard remedy in streaming libraries is a running affine standardization: a `StandardScaler` for the features and, in **river** [Montiel et al., 2021], a `TargetStandardScaler` for the target. This paper measures, and then partially proves, the value of a stronger preprocessing contract: give every numeric stream its own online *distributional* forecaster, and hand the learner the forecaster's state instead of the raw value.

Concretely, each stream contributes two scalars per tick. The first is the one-step predictive mean, what the forecaster expected this value to be. The second is the standardized surprise

$$\bar{z}_t \;=\; \Phi^{-1}\big(\mathrm{clip}(\hat{F}_t(y_t),\, \varepsilon,\, 1-\varepsilon)\big), \qquad \varepsilon = 10^{-12}, \tag{1}$$

the probability integral transform [Rosenblatt, 1952] of the arriving value under the predictive CDF $\hat{F}_t$ issued for it, pushed through the standard normal quantile and clamped so $|\bar{z}_t| \leq z_{\max} = \Phi^{-1}(1-\varepsilon) \approx 7.03$. The mean carries the level; the surprise carries the news. A wild observation can move the pair only so far, and that bounded influence is where the robustness comes from. When the forecaster is calibrated the surprises are approximately independent standard normal, so the learner receives variables that are stationary in scale and carry, by construction, only what the forecaster could not explain.

The idea is the regression instance of a front-end thesis established elsewhere for anomaly detectors and for forecasters: other methods improve when run in the coordinates of a calibrated online forecaster [Cotton, 2026]. The forecaster used throughout is `laplace` from the **skaters** package, a likelihood-weighted ensemble over conjugated transform chains, though nothing in the construction or the theorem depends on that choice.

The paper makes three contributions. First, a package: **ice-skaters** ships `LaplaceFeatures`, a **river** transformer for input streams, and `LaplaceTarget`, a regressor wrapper that adds the target's own pair, both pipe-, pickle- and deep-copy-compatible with the **river** estimator protocol (Section 2). Second, evidence: a three-part benchmark study, contaminated simulation, **river**'s learners against **river**'s recommended pipeline, and **river**'s own datasets, followed by an ablation that decomposes the pair and reverses part of the initial recommendation (Section 3). Third, a theorem: pathwise regret transfer with measured constants and no calibration assumption anywhere (Section 4), whose insurance corollary resolves the one catastrophic failure mode the study observed.

2 The package

Installation is `pip install ice-skaters`; dependencies are **skaters** and **river** only. The full recipe reads:

```
from river import linear_model, preprocessing
from ice_skaters import LaplaceFeatures, LaplaceTarget

model = LaplaceTarget(
    regressor=preprocessing.TargetStandardScaler(
        regressor=LaplaceFeatures()
        | preprocessing.StandardScaler()
        | linear_model.LinearRegression()))
```

`LaplaceFeatures` maintains one forecaster per numeric feature key and replaces the value of key `k` with `mu_k` and `z_k` as above. Non-numeric values pass through untouched. Non-finite numeric values are imputed by the forecast itself, the predictive mean with $z = 0$ ("expected value, no news"), and do not advance the forecaster; a running scaler, by contrast, propagates NaN. `LaplaceTarget` wraps any regressor in the style of `TargetStandardScaler` and augments the features with the target's own pair, which a transformer cannot supply because it never sees the target; the target itself is never transformed.

Three implementation contracts matter in practice. *Alignment*: **river**'s `Pipeline.learn_one` updates unsupervised transformers before transforming for downstream steps, which would hand the model fresher features at learn time than it predicted with; `LaplaceFeatures` caches the pre-update feature dictionary during `learn_one` and serves it to the immediately following `transform_one`, so the predict and learn paths see identical features and

scenario	raw	zscore	robust	huber	zin
clean	**0.001**	0.036	0.108	0.001	0.057
spikes on x	1.08	0.98	0.91	1.38	**0.72**
spikes on y	3.78	5.66	4.42	3.75	**1.84**
spikes on both	9.05	9.78	7.66	9.65	**6.33**
heavy $t(2)$	0.008	2.05	2.17	**0.006**	0.95
drift	**0.003**	0.059	0.232	0.003	0.059
distortion	1.05	0.71	0.53	1.48	**0.51**
shift	**0.001**	0.105	0.159	0.001	0.089

Table 1: Excess MSE against the clean conditional mean, medians over 30 seeds. `zin` beats raw 30/30 seeds on every contaminated scenario and beats the purpose-built robust baseline on all of them.

each forecaster advances exactly once per sample. *Serialization*: instances hold only plain state, never closures, so both estimators pickle and deep-copy. *Robustness*: observations are winsorized before a forecaster consumes them, at 10^{60} absolute plus a window twelve orders of magnitude above the current predictive level; the window is magnitude-relative rather than sigma-relative, because after a degenerate-variance stretch (missing-data zeros, say) a legitimate value sits billions of sigmas out and must pass. The gate is exact identity on any stream double arithmetic can represent comfortably.

The measured cost is about 390 microseconds per stream per sample single-threaded, roughly 900× **river**'s `StandardScaler`, or about 2,500 samples per second per stream. That is the price of a full distributional forecaster per stream; it is right for polls, sensors, and market bars, and wrong inside a hot path at hundreds of thousands of ticks per second.

3 Evidence

Protocols, harnesses and complete tables live in the companion results log;[1] this section reports the headline cells and every boundary.

3.1 Contaminated simulation: coordinates, not the loss

One fixed recursive-least-squares learner with forgetting predicts y one step ahead on a linear Gaussian pair (x AR(0.8) driving y); only the coordinate system varies. Eight scenarios, thirty seeds; the metric is excess mean squared error against the clean conditional mean (the oracle is the zero point), medians over seeds. Table 1 compares raw coordinates, a running z-score (the RevIN-style affine case, Kim et al., 2021), a median/MAD winsorizer, a Huberised update [Huber, 1964], and the two-scalar substitution (`zin`).

Two losing rows matter as much as the wins. With genuine $t(2)$ driving noise and a correctly specified model, the extreme points are the most informative ones and every coordinate fix pays for taming the signal: whether the tails are noise or signal decides the coordinates. And the Huber column shows the loss is the wrong place to fix inputs, since clipping a residual cannot repair a corrupted feature.

[1] `https://github.com/microprediction/timemachines`, `benchmarks/RESULTS.md` section 6; five resumable harnesses with fixed seeds.

dataset	**river** pipeline	+ target pair (`lt`)	forecaster alone
TrumpApproval	0.334	**0.301**	0.150
ChickWeights	**23.8**	24.1	25.5
AirlinePassengers	41.9	**26.6**	29.4
Bikes (20k)	5.07	**5.01**	4.94

Table 2: MAE, progressive validation, `HoeffdingTreeRegressor`, data untouched. `lt` is `LaplaceTarget` alone: raw features plus the target's (mean, surprise) pair. The last column is the target's predictive mean with no regression at all, the attribution control.

3.2 river's learners and river's datasets

The pattern survives transplanting to third-party learners against a third-party baseline. With **river**'s `LinearRegression` and `HoeffdingTreeRegressor` at defaults, and **river**'s recommended `StandardScaler` + `TargetStandardScaler` pipeline as the baseline, the front-end composed with that same pipeline wins 30/30 seeds under feature spikes and 27–30/30 under target spikes for the linear learner (sign test $p \leq 1.9 \times 10^{-9}$), with the tree learner close behind. `KNNRegressor` is a genuine counterexample: neighbour averaging is already spike-robust and the extra dimensions degrade its distance metric.

On **river**'s own datasets (progressive validation, MAE, numeric features, data untouched), the attribution control changes the story (Table 2): on history-dominated streams the univariate forecaster of the target *alone*, no regression, no features, already beats the full feature pipeline, on **river**'s flagship documentation example by 2.2×. The features add almost nothing the calibrated forecaster had not extracted. Where features carry entity identity that one forecaster per key cannot represent (`ChickWeights` interleaves fifty growth curves), the pipeline keeps its edge.

3.3 The ablation, and the revised recommendation

Decomposing the pair on the simulation: the surprise alone is useless (excess MSE 17 to 146; surprises carry no level, and a level cannot be regressed from pure news), the mean alone pays a large clean toll and misses distortion, and together they recombine into a conditional mean neither scalar supports alone. On **river**'s datasets the sharpest cell is the one the first pass missed and Table 2 reports: `LaplaceTarget` alone, raw features untouched, beats the pipeline on three of four datasets untouched, wins ten seeds of ten under 2% feature spikes on those three despite its features being the raw, spiked ones, and ties the counterexample. The target pair anchors the prediction and reduces reliance on corrupted features.

The recommendation therefore orders itself: add the target pair always, one wrapper, helps clean and contaminated alike; replace input features with their pairs only when the features themselves are distrusted; never use the surprise alone.

4 Theory: pathwise regret transfer

The empirical study contains one catastrophic cell: learning against the target's own surprise and mapping point predictions back through a contaminated forecaster amplified target spikes by 60×. The theorem below explains both why the density version of that construction is nevertheless safe on any data whatsoever, and why the front-end's boundedness buys worst-case guarantees no raw-coordinate method has. Throughout, nothing is assumed about the data or about forecast quality; the reader may imagine an adversary choosing $y_{1:T}$.

4.1 Setup

A *body* is any causal algorithm that, having seen $y_1, \dots, y_{t-1}$, emits a predictive distribution for y_t with continuous, strictly increasing CDF $\hat{F}_t$ and strictly positive density $\hat{f}_t$; Gaussian mixtures, hence every skater, qualify. The unclamped surprise is $z_t = \Phi^{-1}(\hat{F}_t(y_t))$ and the clamped surprise $\bar{z}_t$ is Equation 1. Let $\phi_t \in \mathbb{R}^d$ be any causal features with $\|\phi_t\|_\infty \le z_{\max}$, for instance a constant together with clamped surprises of exogenous streams and of the target's own past, which is what the package emits.

Lemma 1 (Conjugation identity)**.** *For any density q on $\mathbb{R}$,*

$$p_q(y) \;=\; \hat{f}_t(y)\,\frac{q(z_t(y))}{\varphi(z_t(y))}, \qquad z_t(y) = \Phi^{-1}(\hat{F}_t(y)),$$

is a probability density, and $\log p_q(y_t) = \log \hat{f}_t(y_t) + \log q(z_t) - \log \varphi(z_t)$.

Proof. Substituting $z = z_t(y)$, whose derivative is $dz/dy = \hat{f}_t(y)/\varphi(z)$, gives $\int p_q = \int q = 1$; the identity is the definition evaluated at y_t. □

The identity is pathwise and exact for the actual, prequentially estimated $\hat{F}_t$ [Dawid, 1984], however miscalibrated; forecast quality never enters. The refinement class is Gaussian, $q_m = \mathcal{N}(m, s^2)$ for fixed $s > 0$. The *sandwich* uses $m_t = \hat{z}_t$, the prediction of the forward (Vovk–Azoury–Warmuth) forecaster [Vovk, 2001, Azoury and Warmuth, 2001, Cesa-Bianchi and Lugosi, 2006] trained on the clamped surprises: with $A_t = \lambda I + \sum_{s\le t} \phi_s \phi_s'$ and $b_{t-1} = \sum_{s<t} \bar{z}_s \phi_s$, it predicts $\hat{z}_t = \phi_t' A_t^{-1} b_{t-1}$. Note A_t includes the current ϕ_t; that forward-looking regularization is the Azoury–Warmuth device.

4.2 The theorem

Theorem 1 (Pathwise regret transfer)**.** *For every sequence $y_{1:T}$, every comparator $u \in \mathbb{R}^d$, and with $C = \{t : \hat{F}_t(y_t) \notin [\varepsilon, 1-\varepsilon]\}$ the set of clamp-active ticks,*

$$\sum_{t=1}^{T} \Big[\log p_{q_{u'\phi_t}}(y_t) - \log p_{q_{\hat{z}_t}}(y_t)\Big] \;\le\; \frac{1}{2s^2}\Big[\lambda\|u\|^2 + z_{\max}^2 \ln\frac{\det A_T}{\lambda^d}\Big] + \frac{1}{s^2}\sum_{t\in C}(u'\phi_t - \hat{z}_t)(z_t - \bar{z}_t),$$

and $\ln(\det A_T/\lambda^d) \le d\ln(1 + Tz_{\max}^2/\lambda)$ *under the feature bound. Every quantity on the right-hand side is computable from the run: λ, s and $z_{\max}$ are chosen,* $\det A_T$ *and the clamp-correction sum are observed, and C consists of ticks the body assigned probability below* 10^{-12}.

Proof. By Lemma 1, for any two refinement means a, b the log-score difference at tick t is $[(z_t - b)^2 - (z_t - a)^2]/2s^2$, so the left-hand side equals $\frac{1}{2s^2}\sum_t[(z_t - \hat{z}_t)^2 - (z_t - u'\phi_t)^2]$. Since $g(z) = (z - \hat{z}_t)^2 - (z - u'\phi_t)^2$ is affine in z with slope $2(u'\phi_t - \hat{z}_t)$,

$$(z_t - \hat{z}_t)^2 - (z_t - u'\phi_t)^2 = (\bar{z}_t - \hat{z}_t)^2 - (\bar{z}_t - u'\phi_t)^2 + 2(u'\phi_t - \hat{z}_t)(z_t - \bar{z}_t),$$

and the correction vanishes off C where $z_t = \bar{z}_t$. The clamped sum is the squared-loss regret of the forward forecaster on targets bounded by $z_{\max}$, which Appendix A bounds by $\lambda\|u\|^2 + z_{\max}^2 \ln(\det A_T/\lambda^d)$. The determinant bound is $\det A_T \le (\operatorname{tr} A_T/d)^d$ with $\operatorname{tr} A_T \le \lambda d + Tdz_{\max}^2$. □

Corollary 1 (Insurance)**.** *Take $s = 1$ and $u = 0$. Then $q_0 = \varphi$ and $p_{q_0} = \hat{f}_t$ is the body itself, so for every data sequence*

$$\sum_t \log \hat{f}_t(y_t) - \sum_t \log p_{q_{\hat{z}_t}}(y_t) \;\le\; \frac{z_{\max}^2}{2}\, d \ln\Big(1 + \frac{T z_{\max}^2}{\lambda}\Big) + \sum_{t \in C} 2\, |\hat{z}_t|\, |z_t - \bar{z}_t|.$$

The sandwich can never lose more than $O(d \log T)$ nats to the body, pathwise, while gaining without limit whatever linear structure exists in the surprises.

Remark 1. The corollary resolves the study's catastrophic cell. The output sandwich looked disastrous there, but that failure was a property of extracting a point forecast, the mean of the pushforward, whose tails a contaminated body inflates. As a density, scored where densities are scored, the sandwich is safe on any data by Corollary 1. Fix the inputs for point prediction; trust the sandwich only in log-loss.

4.3 The oracle decomposition

Only now suppose a truth: (x_t, y_t) adapted to a filtration $\mathcal{G}_t$, with P_t the conditional law of y_t given $\mathcal{G}_{t-1}$, and let the body be causal in y alone with $\mathcal{F}_{t-1} = \sigma(y_{1:t-1})$. Let Z_t^* and $\bar{Z}_t$ denote the conditional laws of z_t given $\mathcal{G}_{t-1}$ and $\mathcal{F}_{t-1}$ respectively.

Proposition 1 (Transfer entropy plus measured miscalibration)**.**

$$\mathbb{E} \sum_t \mathrm{KL}(P_t \,\|\, \hat{F}_t) \;=\; \sum_t I\big(y_t;\, \mathcal{G}_{t-1} \mid \mathcal{F}_{t-1}\big) \;+\; \mathbb{E} \sum_t \mathrm{KL}\big(\bar{Z}_t \,\|\, \mathcal{N}(0,1)\big).$$

Proof. KL divergence is invariant under an invertible transformation applied to both arguments; applying $y \mapsto \Phi^{-1}(\hat{F}_t(y))$, which is $\mathcal{F}_{t-1}$-measurable and strictly increasing, maps P_t to Z_t^* and $\hat{F}_t$ to $\mathcal{N}(0,1)$, so $\mathrm{KL}(P_t \| \hat{F}_t) = \mathrm{KL}(Z_t^* \| \mathcal{N}(0,1))$. The chain rule of relative entropy against a fixed reference splits, in expectation over $\mathcal{G}_{t-1}$, $\mathbb{E}\,\mathrm{KL}(Z_t^* \| \mathcal{N}(0,1)) = \mathbb{E}\,\mathrm{KL}(Z_t^* \| \bar{Z}_t) + \mathbb{E}\,\mathrm{KL}(\bar{Z}_t \| \mathcal{N}(0,1))$, and the first term is the conditional mutual information after the same invariance is applied once more. ☐

The body's expected excess over the full-information oracle splits exactly into the transfer entropy of the exogenous information and the body's own prequential miscalibration, measured as the divergence of the actual conditional surprise law from standard normal. The refinement learner of Theorem 1 competes for the first term only; the second is the body's business, and it is a measured quantity, never an assumption.

4.4 What is not claimed, and the numerical check

No claim that the surprises are independent, standard normal, or stationary. No claim about point predictions on the original scale; Section 3 measured where those go wrong. The comparator class is the conjugated linear-Gaussian family, not all forecasters; its force is that it contains the body itself and every linear refinement of its surprises, with constants a run can print. A test in the package repository runs the exact construction, Laplace bodies, clamped surprises, forward learner, on clean and spiked streams and asserts the theorem's inequality for a grid of comparators; the bound becomes informative (smaller than the trivial total loss) near $T \approx 2{,}500$ for $d = 3$.

5 Related work

Reversible instance normalization and its relatives [Kim et al., 2021, Passalis et al., 2019] remove per-instance mean and variance around a deep forecaster and add them back: the affine, window-based special case of the present construction, welded to a specific model class. The nonparanormal [Liu et al., 2009] estimates monotone marginal transforms so that a Gaussian model fits in transformed space; the front-end is its dynamic analogue, with conditional, time-varying margins supplied by a forecaster rather than unconditional ranks. Robust streaming regression modifies the loss [Huber, 1964]; Table 1 argues the coordinates are the better lever, since a clipped residual cannot repair a corrupted feature. The regret machinery is the forward algorithm of Vovk [2001] and Azoury and Warmuth [2001] as presented by Cesa-Bianchi and Lugosi [2006]; the contribution here is the transfer through the conjugation and the observation that the parade clamp manufactures the boundedness those results assume, turning an assumption into a property.

6 Limitations and next steps

The boundaries are stated in Section 3: distance-based learners, entity-interleaved streams, informative heavy tails, and the 900× per-tick cost. Open items, in order: per-entity bodies with partial pooling in surprise space (the `ChickWeights` fix); multi-horizon surprises from $k > 1$ parades for drift and shift; and concept-drift classification. The package and all harnesses are open source; every table in this paper regenerates from fixed seeds.

A The forward-algorithm bound

Proposition 2 (Vovk, 2001, Azoury and Warmuth, 2001; Cesa-Bianchi and Lugosi, 2006, Theorem 11.8)**.** *For targets $|\bar{z}_t| \le Y$ and predictions $\hat{z}_t = \phi_t' A_t^{-1} b_{t-1}$ as above,*

$$\sum_t (\hat{z}_t - \bar{z}_t)^2 - \sum_t (u'\phi_t - \bar{z}_t)^2 \;\le\; \lambda \|u\|^2 + Y^2 \ln \frac{\det A_T}{\lambda^d} \qquad \textit{for all } u.$$

Proof sketch. Let $w_t = A_t^{-1} b_t$ and define the potential $\Phi_t = \min_w [\lambda\|w\|^2 + \sum_{s \le t} (w'\phi_s - \bar{z}_s)^2]$, attained at w_t. Per-tick algebra expresses $(\hat{z}_t - \bar{z}_t)^2$ as $(\Phi_t - \Phi_{t-1})$ plus an extra term that telescopes against $\bar{z}_t^2\, \phi_t' A_t^{-1} \phi_t$; summing,

$$\sum_t (\hat{z}_t - \bar{z}_t)^2 \;\le\; \Phi_T + \sum_t \bar{z}_t^2\, \phi_t' A_t^{-1} \phi_t \;\le\; \lambda\|u\|^2 + \sum_t (u'\phi_t - \bar{z}_t)^2 + Y^2 \sum_t \phi_t' A_t^{-1} \phi_t.$$

Finally $\phi_t' A_t^{-1} \phi_t = 1 - \det A_{t-1} / \det A_t \le \ln(\det A_t / \det A_{t-1})$, using $1 - 1/r \le \ln r$, and the sum telescopes to $\ln(\det A_T / \lambda^d)$. □

References

Katy S. Azoury and Manfred K. Warmuth. Relative loss bounds for on-line density estimation with the exponential family of distributions. *Machine Learning*, 43(3):211–246, 2001.

Nicolò Cesa-Bianchi and Gábor Lugosi. *Prediction, Learning, and Games.* Cambridge University Press, 2006.

Peter Cotton. Transforms all the way down: Automatic online distributional forecasting by conjugation. Working paper, https://github.com/microprediction/skaters/blob/main/papers/skaters-jss.pdf, 2026.

A. Philip Dawid. Present position and potential developments: some personal views. Statistical theory: the prequential approach. *Journal of the Royal Statistical Society, Series A*, 147(2):278–292, 1984.

Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964.

Taesung Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. Reversible instance normalization for accurate time-series forecasting against distribution shift. In *International Conference on Learning Representations*, 2021.

Han Liu, John Lafferty, and Larry Wasserman. The nonparanormal: semiparametric estimation of high dimensional undirected graphs. *Journal of Machine Learning Research*, 10:2295–2328, 2009.

Jacob Montiel, Max Halford, Saulo Martiello Mastelini, Geoffrey Bolmier, Raphael Sourty, Robin Vaysse, Adil Zouitine, Heitor Murilo Gomes, Jesse Read, Talel Abdessalem, and Albert Bifet. River: machine learning for streaming data in Python. *Journal of Machine Learning Research*, 22(110):1–8, 2021.

Nikolaos Passalis, Anastasios Tefas, Juho Kanniainen, Moncef Gabbouj, and Alexandros Iosifidis. Deep adaptive input normalization for time series forecasting. *IEEE Transactions on Neural Networks and Learning Systems*, 31(9):3760–3765, 2019.

Murray Rosenblatt. Remarks on a multivariate transformation. *The Annals of Mathematical Statistics*, 23(3):470–472, 1952.

Vladimir Vovk. Competitive on-line statistics. *International Statistical Review*, 69(2):213–248, 2001.